KIẾN TRÚC VI PHẦN CỨNG PROCESSING ELEMENT (PE UNIT)

Bộ Tăng tốc Mạng Nơ-ron Modified ResNet-18 trên FPGA Xilinx Zynq-7020

Đặc tả Kỹ thuật Thiết kế RTL • Phòng Nghiên cứu Bán dẫn • Tháng 11/2026

1. Tổng quan và Nguyên lý Hoạt động

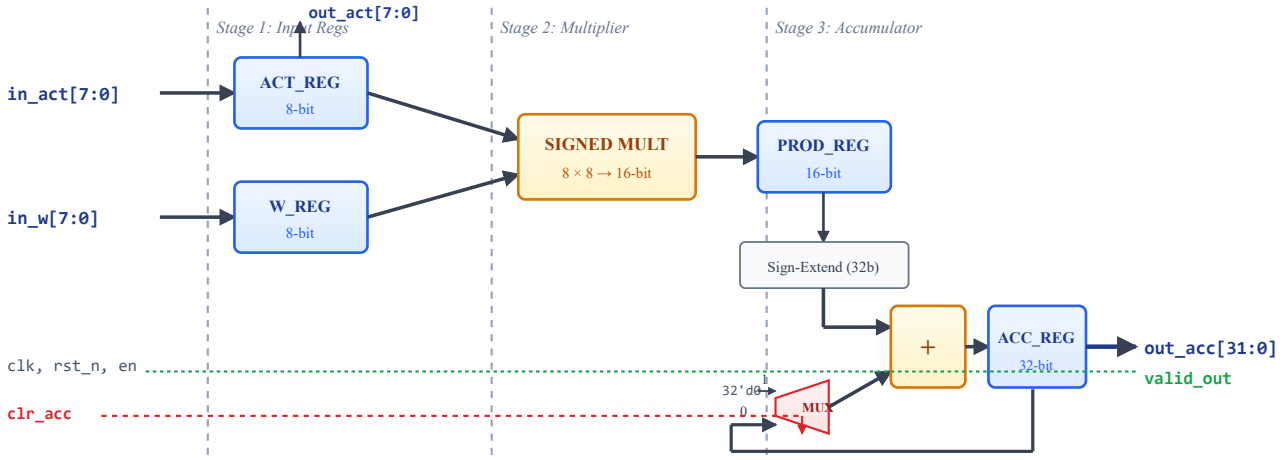
Khối **Processing Element (PE)** là hạt nhân tính toán cơ sở trong mảng tính toán hai chiều `T_m × T_n = 16 × 16` (gồm 256 PEs) của bộ tăng tốc **Modified ResNet-18**. Nhiệm vụ của mỗi PE là thực thi chuỗi phép toán nhân-tích lũy (Multiply-Accumulate — MAC) số nguyên có dấu:

> • Khi **clr_acc = 1** (Bắt đầu Tile mới): `ACC_REG ⇐ in_act × in_w`
> • Khi **clr_acc = 0 và en = 1** (Cộng dồn): `ACC_REG ⇐ ACC_REG + (in_act × in_w)`

- **Activation (`in_act`):** Số nguyên có dấu 8-bit (Two's Complement INT8), miền giá trị `[-128, 127]`.
- **Trọng số (`in_w`):** Số nguyên có dấu 8-bit (Two's Complement INT8), miền giá trị `[-127, 127]`.
- **Tích số (`r_prod`):** Số nguyên có dấu 16-bit (Signed INT16), miền giá trị `[-16,256, 16,384]`.
- **Bộ tích lũy (`out_acc`):** Số nguyên có dấu 32-bit (Signed INT32), đảm bảo không tràn số (chứa tới 65,536 lần cộng dồn).

2. Sơ đồ Kiến trúc Chi tiết Khối PE (Detailed Block Diagram)

Sơ đồ vi kiến trúc dưới đây mô tả chính xác từng thanh ghi đồng bộ, cổng giao tiếp, bộ nhân và bộ tích lũy bên trong module `pe_unit.v`:

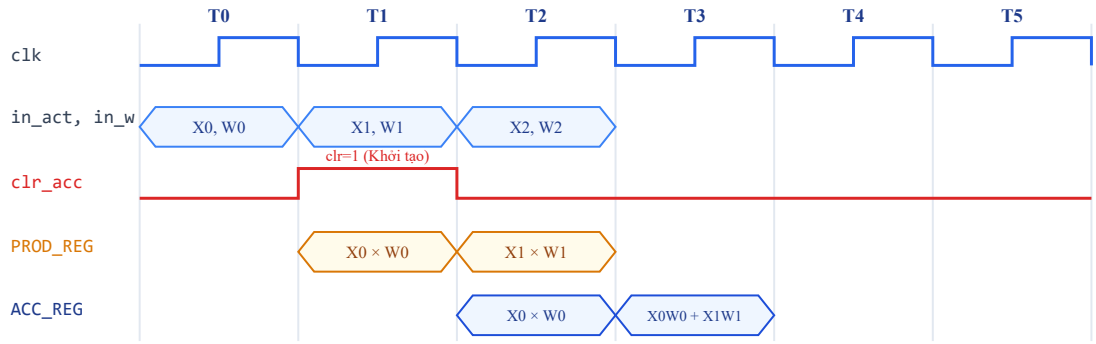


3. Bảng Đặc tả Chân Tín hiệu (Pinout Interface Specification)

Tên Cổng	Hướng	Độ rộng	Kiểu dữ liệu	Chức năng Kỹ thuật Chi tiết
`clk`	Input	1-bit	wire	Xung nhịp hệ thống toàn mạch (Tần số mục tiêu: 100MHz – 150MHz).
`rst_n`	Input	1-bit	wire	Reset bất đồng bộ tích cực mức thấp (Asynchronous Active-Low).
`en`	Input	1-bit	wire	Clock Enable: Cho phép tính toán. Khi `en=0`, giữ nguyên trạng thái thanh ghi.
`clr_acc`	Input	1-bit	wire	**Khởi tạo Tile mới:** Khi `1`, chốt tích số mới vào `ACC_REG`; khi `0`, cộng dồn tiếp.
`in_act[7:0]`	Input	8-bit	signed wire	Dữ liệu điểm ảnh kích hoạt đầu vào (X). Định dạng bù 2 (Signed INT8).
`in_w[7:0]`	Input	8-bit	signed wire	Trọng số mạng nơ-ron (W). Định dạng bù 2 (Signed INT8).
`out_act[7:0]`	Output	8-bit	signed reg	Điểm ảnh chuyển tiếp sang PE kế bên (hỗ trợ truyền dữ liệu trong mảng 16 × 16).
`out_acc[31:0]`	Output	32-bit	signed reg	Kết quả tổng tích lũy INT32 chuyển tiếp sang khối Re-quantization (Scale & Shift).
`valid_out`	Output	1-bit	reg	Báo hiệu kết quả `out_acc` hợp lệ (sau đúng 2 chu kỳ trễ đường ống).

4. Giản đồ Định thời Đường ống (Pipeline Waveform Timing)

Mỗi phép tính tích chập hoàn tất qua đường ống 2 chu kỳ (2-Cycle Pipeline Latency) giúp tối đa hóa tần số cực đại F_{max}:



5. Ánh xạ Tối ưu vào DSP48E1 (Xilinx Zynq-7020)

Khi tổng hợp trên **Vivado Design Suite**, cấu trúc của `pe_unit` được ánh xạ trực tiếp 1-1 vào khối phần cứng chuyên dụng **DSP48E1**:

- Cổng **A (25-bit)** nhận `in_act[7:0]` (mở rộng dấu 8-bit lên 25-bit).
- Cổng **B (18-bit)** nhận `in_w[7:0]` (mở rộng dấu 8-bit lên 18-bit).
- Bộ nhân 25 × 18 nội tại thực hiện nhân hai số có dấu; bộ cộng 48-bit nội tại thực hiện tích lũy $P = P + C$.
- Khai thác toàn bộ thanh ghi nội (`AREG=1, BREG=1, MREG=1, PREG=1`) giúp giải phóng hoàn toàn tài nguyên LUT/FF ngoài fabric, hạn chế tối đa suy hao công suất động.

6. Mã Nguồn Verilog RTL Hoàn Chỉnh (`hardware/RTL/pe_unit.v`)

Khung mã nguồn Verilog HDL chuẩn công nghiệp, hoàn toàn tương thích và sẵn sàng tổng hợp (Synthesizable RTL):

```
// ============================================================================
// Module: pe_unit.v
// Architecture: 2-Cycle Pipelined INT8 Signed MAC Unit with INT32 Accumulator
// Target Device: Xilinx Zynq-7020 (DSP48E1 Optimized)
// ============================================================================
`timescale 1ns / 1ps

module pe_unit (
    input  wire               clk,        // Xung nhip he thong (100 - 150MHz)
    input  wire               rst_n,      // Reset bat dong bo tich cuc muc thap
    input  wire               en,         // Clock enable
    input  wire               clr_acc,    // Khoi tao tile moi (xoa bo tich luy)
    input  wire signed [7:0]  in_act,     // Activation INT8 co dau [-128, 127]
    input  wire signed [7:0]  in_w,       // Weight INT8 co dau [-127, 127]

    output reg  signed [7:0]  out_act,    // Chuyen tiep activation sang PE ke ben
    output reg  signed [31:0] out_acc,    // Ket qua tich luy INT32
    output reg                valid_out   // Bao hieu out_acc hop le sau 2 chu ky
);

    // --- PIPELINE REGISTERS ---
    reg signed [7:0]  r_act;
    reg signed [7:0]  r_w;
    reg signed [15:0] r_prod;
    reg               val_pipe_0;
    reg               val_pipe_1;
    reg               clr_pipe;

    // --- STAGE 1: INPUT REGISTRATION & SYSTOLIC FEEDTHROUGH ---
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_act      <= 8'sd0;
            r_w        <= 8'sd0;
            out_act    <= 8'sd0;
            clr_pipe   <= 1'b0;
            val_pipe_0 <= 1'b0;
        end else if (en) begin
            r_act      <= in_act;
            r_w        <= in_w;
            out_act    <= in_act;          // Chuyen tiep sang PE hang xom
            clr_pipe   <= clr_acc;
            val_pipe_0 <= 1'b1;
        end else begin
            val_pipe_0 <= 1'b0;
        end
    end

    // --- STAGE 2: SIGNED MULTIPLICATION (INT8 x INT8 -> INT16) ---
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_prod     <= 16'sd0;
            val_pipe_1 <= 1'b0;
        end else if (en) begin
            r_prod     <= r_act * r_w;    // Tu dong anh xa vao bo nhan DSP48E1
            val_pipe_1 <= val_pipe_0;
        end else begin
            val_pipe_1 <= 1'b0;
        end
    end

    // --- STAGE 3: 32-BIT ACCUMULATION WITH SIGN-EXTENSION ---
    wire signed [31:0] prod_ext = {{16{r_prod[15]}}, r_prod}; // Mo rong dau 16 -> 32 bit

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            out_acc   <= 32'sd0;
            valid_out <= 1'b0;
        end else if (en) begin
            if (clr_pipe) begin
                out_acc <= prod_ext;           // Nap tich so dau tien cua Tile
            end else begin
                out_acc <= out_acc + prod_ext;  // Tich luy cong don
            end
            valid_out <= val_pipe_1;
        end else begin
            valid_out <= 1'b0;
        end
    end

endmodule
```